CRUSTPairs: A Cross-Language Dataset of Paired C and Rust Solutions

Leonardo J. Almeida
University of São Paulo
leonardo.j.almeida@usp.br

Vojtěch Hejlek
TODO affiliation
TODO email

***Abstract*—C-to-Rust migration is an active agenda, but no dataset pairs the two languages at the level of whole solutions to a shared problem. We present CRUSTPairs, 1857 C/Rust implementation pairs drawn from competitive programming archives, normalised, deduplicated, and stratified into difficulty tiers by zero-shot cross-language embedding similarity, with per-record provenance and licensing. Cross-language retrieval is hard on this data, and harder than a single baseline would suggest: TF-IDF over shared identifiers reaches MRR 0.21 and so beats a pretrained code model at 0.19, which on the hardest quarter of the data scores $1.7\times$ chance. Finetuning reaches 0.79, and lifts that hardest quarter $10\times$. Renaming every user identifier settles what is being learned: the finetuned models keep 97% of their accuracy without them, while the pretrained model keeps 66% — less than the lexical baseline does — and on structurally divergent pairs the finetuned models are *better* without identifiers than with them.**

***Index Terms*—dataset, cross-language, code similarity, Rust, C, code retrieval**

I. Introduction

Memory-safety defects remain the dominant vulnerability class in C. Microsoft has for two decades attributed roughly 70% of its CVEs to that class,[1] and Android shows the substitution works: its annual memory-safety vulnerabilities fell from 223 in 2019 to under 50 in 2024 as Rust adoption grew, with roughly $1000\times$ lower defect density in Rust than in C and C++.[2] Policy has followed, through CISA's memory-safe roadmaps and the US cyber director's call to adopt memory-safe languages;[3] DARPA's TRACTOR programme[4] funds the automated translation of legacy C to Rust outright, on the premise that the codebases that matter are too large to port by hand.

Translation tools are consequently proliferating, and they are evaluated almost entirely on whether their output compiles and passes the original test suite [1], [2]. That is the right primary criterion. It is also incomplete — a translation can pass every test and still be an unfaithful rendering of its source — and it is unavailable precisely where it is most needed, since legacy C is what most often lacks tests. The complementary question, *are these two programs, in two languages, the same program?*, is a similarity judgement rather than an execution one, and answering it with a learned model requires paired supervision.

No such corpus exists for this language pair. The parallel code datasets either span many languages at snippet granularity [3], [4] or go deep on a single pair that is not this one [5], and the largest collections are grouped by problem rather than aligned across languages [6], [7]. None of them includes Rust.

We present CRUSTPairs, 1857 pairs of C and Rust implementations of the same competitive programming problem, each with its statement, normalised entry points, per-record provenance and licence, a difficulty tier derived from zero-shot cross-language embedding similarity, and a frozen stratified split.

Measuring on it reframes what the task requires. A lexical baseline—TF-IDF over shared identifiers, no model—beats a pretrained code model zero-shot, because competitive programmers reuse variable names across languages. Renaming every user identifier separates the regimes: the pretrained model loses a third of its accuracy, more than the lexical baseline does, while finetuned models lose almost nothing. Cross-language matching here is learnable, but not from identifiers.

Contributions.

- CRUSTPairs, 1857 C/Rust solution pairs with problem statements, normalised entry points, per-record provenance and licensing.
- A difficulty stratification derived from zero-shot cross-language embedding similarity, independent of the model family used in evaluation, which we show transfers to a different family.
- Retrieval baselines per difficulty tier over a frozen stratified split, including a lexical baseline and an identifier ablation that together establish what the task does and does not reduce to.

II. Related Datasets

Parallel code corpora divide by what they align and how many languages they span. The largest are not parallel at all: CodeNet [6] and POJ-104, distributed with CodeXGLUE [7], group solutions by problem without shipping an aligned bilingual corpus. Of the parallel ones, xCodeEval [8] is broadest at 17 languages; XLCoST [3] and CoST [4] align snippets rather

[1]Reported by Microsoft's Security Response Centre for roughly two decades, and restated by its Azure CTO in a RustConf 2025 keynote.

[2]"Rust in Android: Move Fast and Fix Things," Google Security Blog, 2025.

[3]CISA, "The Case for Memory Safe Roadmaps," December 2023; Office of the National Cyber Director, "Back to the Building Blocks: A Path Toward Secure and Measurable Software," February 2024.

[4]"Eliminating Memory Safety Vulnerabilities Once and For All," DARPA, July 2024. https://www.darpa.mil/news/2024/memory-safety-vulnerabilities

than whole programs across the 7 languages GeeksForGeeks covers; and AVATAR [5] goes deep on Java/Python instead. None includes Rust, which is the gap CRUSTPairs fills and the reason it is one language pair rather than a matrix.

The modelling literature is correspondingly single-pair-poor. Unsupervised program translation was established on C++/Java/Python by TransCoder [9], the pretrained encoders that cross-language retrieval builds on—CodeBERT [10] and UniXcoder [11]—saw little or no Rust in pretraining, and cross-language clone detection has the same shape: CLCDSA [12] matches type-4 clones across Java, C# and Python with syntactic features and API-call similarity, stopping short of Rust.

III. Construction

A. Sources and pairing

CRUSTPairs draws on two competitive programming archives: xCodeEval [8], itself derived from Codeforces, and IBM Project CodeNet [6], derived from AtCoder. Both distribute accepted submissions, so the two implementations of a pair are known to have passed the same official tests. A pair is one problem, one accepted C submission, and one accepted Rust submission.

B. Normalisation and cleaning

Both sides pass through a static analysis pipeline that strips comments, removes unused functions and macros, normalises encoding, and applies the language's standard formatter. Entry points are renamed to `solution` in both languages so the function name carries no signal—which also means the distributed code no longer compiles standalone (Section VII).

From an initial 2013 pairs, 127 were removed: 2 confirmed mis-pairs, 69 AtCoder easy/hard variants with near-identical statements (description embedding similarity > 0.90), 54 further suspicious pairs (0.80–0.90) found the same way with OpenAI `text-embedding-3-large`, 1 length-ratio outlier and 1 case of genuine implementation divergence. A further 29 textbook reference implementations were dropped: they carried the only copyleft content and the only records whose halves were under different licences (Section VII), and had no problem statement. This leaves 1857 pairs.

C. Difficulty stratification

Each pair is scored by the cosine similarity between zero-shot `SFR-Embedding-Code-400M_R` embeddings of its C and Rust sides, and binned at the 25th and 75th percentiles into easy, medium and hard (thresholds 0.7856 and 0.8687; similarity mean 0.826, standard deviation 0.058). The raw score is distributed alongside the tier, so the cuts can be redrawn without re-running the model. Pairs a generic code model already embeds close together are near-literal translations; pairs it separates diverge structurally and demand a real cross-language remapping.

SFR rather than UniXcoder [11] on purpose: defining the tiers with the family that is later finetuned would make hard pairs hard by construction, and any per-tier improvement circular. The stratification is also origin-agnostic, since AtCoder and Codeforces rate difficulty on incompatible scales.

TABLE I
SCHEMA. ONE JSON OBJECT PER LINE OF `DATASET.JSONL`; ALL FIELDS ARE STRINGS EXCEPT `DIFFICULTY_SCORE`.

Field	Values
`problem_id`	`0001–1886`, inherited
`origin`	`codenet` (839), `xcodeeval` (1018)
`license`	SPDX expression; follows `origin`
`problem_description`	upstream statement, verbatim; `_format` is `html` (839) or `text` (1018)
`c_code`, `rust_code`	entry points renamed `solution`
`difficulty`	`easy` (465), `medium` (928), `hard` (464); `_score` is the raw similarity
`split`	`train` (1485), `val` (185), `test` (187)

IV. Dataset Description

A. Schema

CRUSTPairs is a single 5.9 MB JSON Lines file, one object per pair, with the fields in Table I. Nothing is split across files and no field needs to be joined from elsewhere.

The fifth row is `problem_description_format`, abbreviated to fit; `problem_id` runs to 1886 over 1857 records, keeping an earlier release's numbering. Two choices are worth naming. `license` is per record, because the sources differ (Section VII), so filtering on `origin` is the only way to extract a commercially usable subset. And `difficulty_score` ships the raw similarity beside the tier, so the quantile cuts can be redrawn without re-running the model.

B. Statistics

Solutions are short: a median of 516 characters of C and 749 of Rust, with problem statements at a median of 1231. The Rust side runs about $1.5\times$ longer in characters (median ratio 1.50) while being *shorter* in lines —31.4 against 33.6 on average—so Rust solutions pack more into each line. A length-based heuristic therefore does not separate the two languages the way the character counts alone would suggest.

Source and difficulty are not independent. CodeNet contributes 241 easy, 408 medium and 190 hard pairs; xCodeEval 224, 520 and 274. The CodeNet portion skews easy while the xCodeEval portion skews hard. Reporting a per-tier result therefore requires a split that controls for both.

C. Splits

The train/validation/test assignment is frozen in the data rather than regenerated from a seed, and is stratified by source $\times$ difficulty at 80/10/10: 1485, 185 and 187 pairs. The test set mirrors the full dataset to within 0.3 percentage points by source and 0.6 by tier.

TABLE II
CROSS-LANGUAGE RETRIEVAL ON THE 187-PAIR TEST SET.

Model	MRR	MRR@10	R@1	R@5	R@10
Chance	0.031	0.016	0.005	0.027	0.053
Lexical TF-IDF	0.206	0.191	0.128	0.273	0.353
UniXcoder, zero-shot	0.189	0.170	0.134	0.203	0.283
UniXcoder + LoRA	0.724	0.720	0.624	0.836	0.913
UniXcoder, full	**0.792**	**0.791**	**0.706**	**0.909**	**0.952**

TABLE III
MRR BY DIFFICULTY TIER, WITH IDENTIFIERS KEPT → RENAMED. EVERY TIER IS SCORED AGAINST THE SAME 187-CANDIDATE POOL; ONLY THE QUERIES ARE PARTITIONED. *Ret.* IS THE ANONYMISED OVERALL MRR AS A FRACTION OF THE NORMAL ONE.

Model	Easy	Medium	Hard	Ret.
Lexical TF-IDF	.446→.355	.144→.094	.089→.080	76%
UniXcoder zero-shot	.328→.276	.186→.084	.053→.048	66%
UniXcoder + LoRA	.957→.941	.776→.688	.388→**.429**	96%
UniXcoder, full	.979→.951	.821→.756	.550→**.611**	97%

V. BASELINES

Retrieval is C-to-Rust unless stated otherwise: each C implementation queries the pool of all 187 Rust implementations in the test split, and the target is the one solving the same problem.

We report mean reciprocal rank, introduced for the TREC-8 question answering track [13] and standard for code search since CodeSearchNet [14] and CodeXGLUE [7]: each query contributes the reciprocal of the rank of its counterpart, so the score turns on whether the answer lands at the top. MRR@10 truncates that, contributing zero beyond rank ten; both are given because they diverge in the zero-shot regime, which is the denominator of every ratio below. Chance is $\mathrm{MRR} = 0.031$ at this pool size. LoRA figures are mean $\pm$ sd over four seeds; other rows are single runs.

The lexical row is the one that fixes the frame. TF-IDF over shared identifiers, with no model at all, reaches 0.206 and so *beats* zero-shot UniXcoder [11] at 0.189. Competitive programmers reuse variable names across languages: `n` is shared by 85 of the 187 test pairs, `i` by 84, then `a`, `t`, `s`, `j`. The claim this dataset supports is therefore not that a generic code model finds the task hard, but the stronger one that *neither* generic code embeddings *nor* lexical matching solve it, and finetuning does — $3.8\times$ over zero-shot for LoRA [15] and $4.2\times$ for full finetuning ($4.2\times$ and $4.7\times$ on MRR@10, which diverges from MRR because many zero-shot positives fall outside the top ten).

A. Difficulty tiers and the identifier ablation

Table III reports MRR per tier, and repeats every configuration with user identifiers renamed to positional names independently in each language, which destroys cross-language name correspondence while leaving control flow, types, library calls and string literals intact.

Three things follow, and together they are the strongest evidence that this dataset measures what it is meant to.

Retrieval is monotone in tier for every configuration. The tiers were cut with SFR while these models are UniXcoder-based, so the ordering is a property of the pairs, not of the encoder that defined it. Zero-shot UniXcoder reaches 0.053 on the hard tier—$1.7\times$ chance, with one of the 47 hard pairs at rank 1. A generic code model effectively cannot solve a quarter of this dataset.

Finetuning learns structure, not names. The finetuned models keep 96–97% of their MRR when identifiers are destroyed; zero-shot UniXcoder keeps 66%, *less* than the lexical baseline's 76%. A pretrained code model is more name-dependent than TF-IDF, so it inverts the natural reading of Table II: zero-shot UniXcoder is not merely failing to beat lexical matching—it is doing a worse version of the same thing.

On hard pairs, identifiers are a liability. Both finetuned models do *better* without them—LoRA 0.388 to 0.429, full finetuning 0.550 to 0.611, both about $+11\%$—while the lexical and zero-shot models get worse. Where implementations diverge structurally, a shared name is more often coincidence than signal.

B. Direction and parameter efficiency

Querying with Rust beats querying with C (MRR 0.189 to 0.233 zero-shot, 0.792 to 0.836 full), and the asymmetry nearly vanishes under the ablation (0.768 against 0.773), locating it in identifiers rather than structure. LoRA reaches 91% of full finetuning overall with 0.23% of the parameters, but 98% on easy pairs against 71% on hard: the budget binds where the remapping is non-trivial, and the aggregate ratio conceals it.

VI. USE CASES

Section I set out why C-to-Rust translation is being funded rather than merely discussed. What follows is the gap between that effort and what it currently measures.

A. What a paired corpus contributes

Translation tools are evaluated almost entirely on whether their output compiles and passes the original tests; TRACTOR's own benchmark is 150 C programs scored for correctness and safety.[5] That is the right primary criterion and an incomplete one—a translation can pass every test and still render its source unfaithfully—and it is unavailable where it is most needed, since legacy C is what most often lacks tests. The complementary question, *are these two programs the same program?*, is a similarity judgement, and it is the one CRUSTPairs supplies supervision for.

Two uses follow. **Scoring translator output without tests**: a model finetuned on CRUSTPairs scores a (C, Rust) pair, flagging translations that drift from their source, with the per-tier results bounding its usable range—0.55 MRR on structurally

[5] "Translating All C to Rust (TRACTOR) Benchmarks," MIT Lincoln Laboratory. https://www.ll.mit.edu/r-d/projects/

divergent pairs, not the 0.98 the easy tier suggests. **Finding an existing port**: equivalent functionality often already exists in a crate, and locating it is cross-language retrieval over whole implementations.

These are competitive programming solutions, without the allocation patterns, aliasing and concurrency that make real C hard to port; read the results as a measurement on a clean proxy rather than evidence about systems code.

VII. Limitations, Availability and Licensing

Equivalence is inherited, not verified. No tests or expected I/O are distributed; that both submissions were accepted upstream is evidence of functional equivalence, not a check of it, and the entry-point renaming means the code does not compile standalone. Execution-based verification is the clearest future work. Relatedly, upstream identifiers are absent, so pairs cannot be traced to the originating submission, and difficulty is a proxy for embedding similarity rather than algorithmic difficulty. These are also competitive programming solutions—short, single-file, I/O-driven—so nothing here should be assumed to transfer to systems code, and pre-training contamination is possible, both archives being public.

Variance is measured for one configuration. The LoRA rows are means over four seeds ($\sigma = 0.011$ on MRR); the others are single runs. Separately, an earlier release's tier labels proved not exactly reproducible: regenerating them matches the aggregate distribution but only 89% of individual labels, with disagreement spread symmetrically across adjacent tiers—which threshold movement alone cannot produce. The released labels are the regenerated ones, with raw scores alongside.

Licensing. CRUSTPairs aggregates two corpora under different licences, so every record carries its own SPDX expression: the 1018 xCodeEval-derived pairs are CC BY-NC 4.0 and the 839 CodeNet-derived pairs are CDLA-Permissive-2.0. The aggregate is therefore non-commercial and requires attribution to both projects; the CodeNet portion alone gives 839 pairs with no commercial restriction.

Acknowledgements

TODO.

References

[1] V. Nitin, R. Krishna, L. Lemos do Valle, and B. Ray, "C2SaferRust: Transforming C projects into safer Rust with neurosymbolic techniques," *IEEE Transactions on Software Engineering*, vol. 52, no. 2, pp. 618–630, 2026.

[2] M. Shetty, N. Jain, A. Godbole, S. A. Seshia, and K. Sen, "Syzygy: Dual code-test C to (safe) Rust translation using LLMs and dynamic analysis," arXiv preprint arXiv:2412.14234, 2024.

[3] M. Zhu, A. Jain, K. Suresh, R. Ravindran, S. Tipirneni, and C. K. Reddy, "XLCoST: A benchmark dataset for cross-lingual code intelligence," arXiv preprint arXiv:2206.08474, 2022.

[4] M. Zhu, K. Suresh, and C. K. Reddy, "Multilingual code snippets training for program translation," in *Proceedings of the 36th AAAI Conference on Artificial Intelligence*, 2022, pp. 11 783–11 790.

[5] W. U. Ahmad, M. G. R. Tushar, S. Chakraborty, and K.-W. Chang, "AVATAR: A parallel corpus for Java-Python program translation," in *Findings of the Association for Computational Linguistics: ACL 2023*, 2023, pp. 2268–2281.

[6] R. Puri, D. S. Kung, G. Janssen, W. Zhang, G. Domeniconi, V. Zolotov, J. Dolby, J. Chen, M. R. Choudhury, L. Decker, V. Thost, L. Buratti, S. Pujar, S. Ramji, U. Finkler, S. Malaika, and F. Reiss, "CodeNet: A large-scale AI for code dataset for learning a diversity of coding tasks," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.

[7] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. B. Clement, D. Drain, D. Jiang, D. Tang, G. Li, L. Zhou, L. Shou, L. Zhou, M. Tufano, M. Gong, M. Zhou, N. Duan, N. Sundaresan, S. K. Deng, S. Fu, and S. Liu, "CodeXGLUE: A machine learning benchmark dataset for code understanding and generation," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.

[8] M. A. M. Khan, M. S. Bari, X. L. Do, W. Wang, M. R. Parvez, and S. Joty, "XCodeEval: An execution-based large scale multilingual multitask benchmark for code understanding, generation, translation and retrieval," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024, pp. 6766–6805.

[9] B. Roziere, M.-A. Lachaux, L. Chanussot, and G. Lample, "Unsupervised translation of programming languages," in *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020.

[10] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1536–1547.

[11] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "UniXcoder: Unified cross-modal pre-training for code representation," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 2022, pp. 7212–7225.

[12] K. W. Nafi, T. S. Kar, B. Roy, C. K. Roy, and K. A. Schneider, "CLCDSA: Cross language code clone detection using syntactical features and API documentation," in *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2019, pp. 1026–1037.

[13] E. M. Voorhees, "The TREC-8 question answering track report," in *Proceedings of the 8th Text REtrieval Conference (TREC-8)*, 1999.

[14] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "CodeSearchNet challenge: Evaluating the state of semantic code search," arXiv preprint arXiv:1909.09436, 2019.

[15] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.